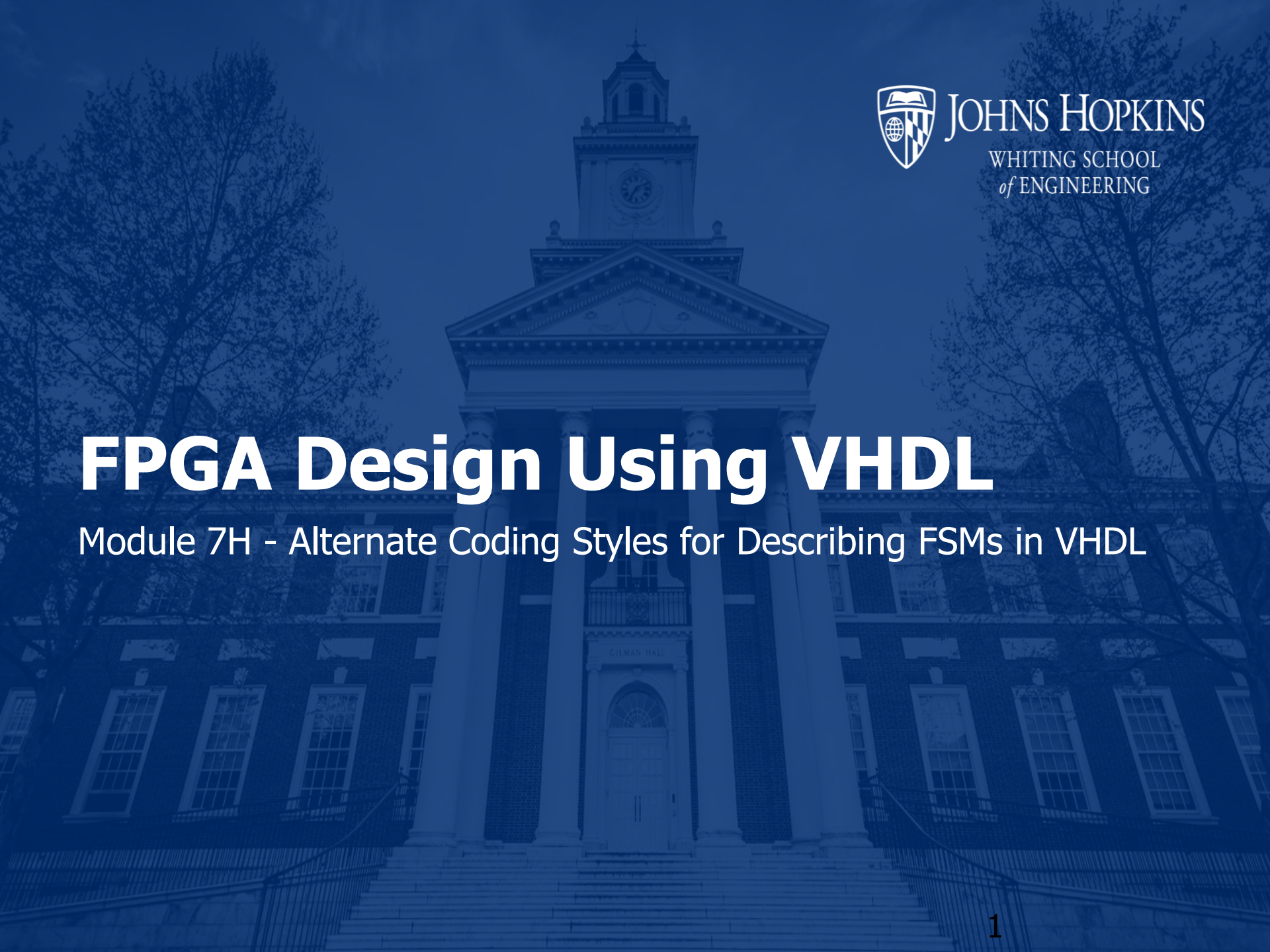


FPGA Design Using VHDL

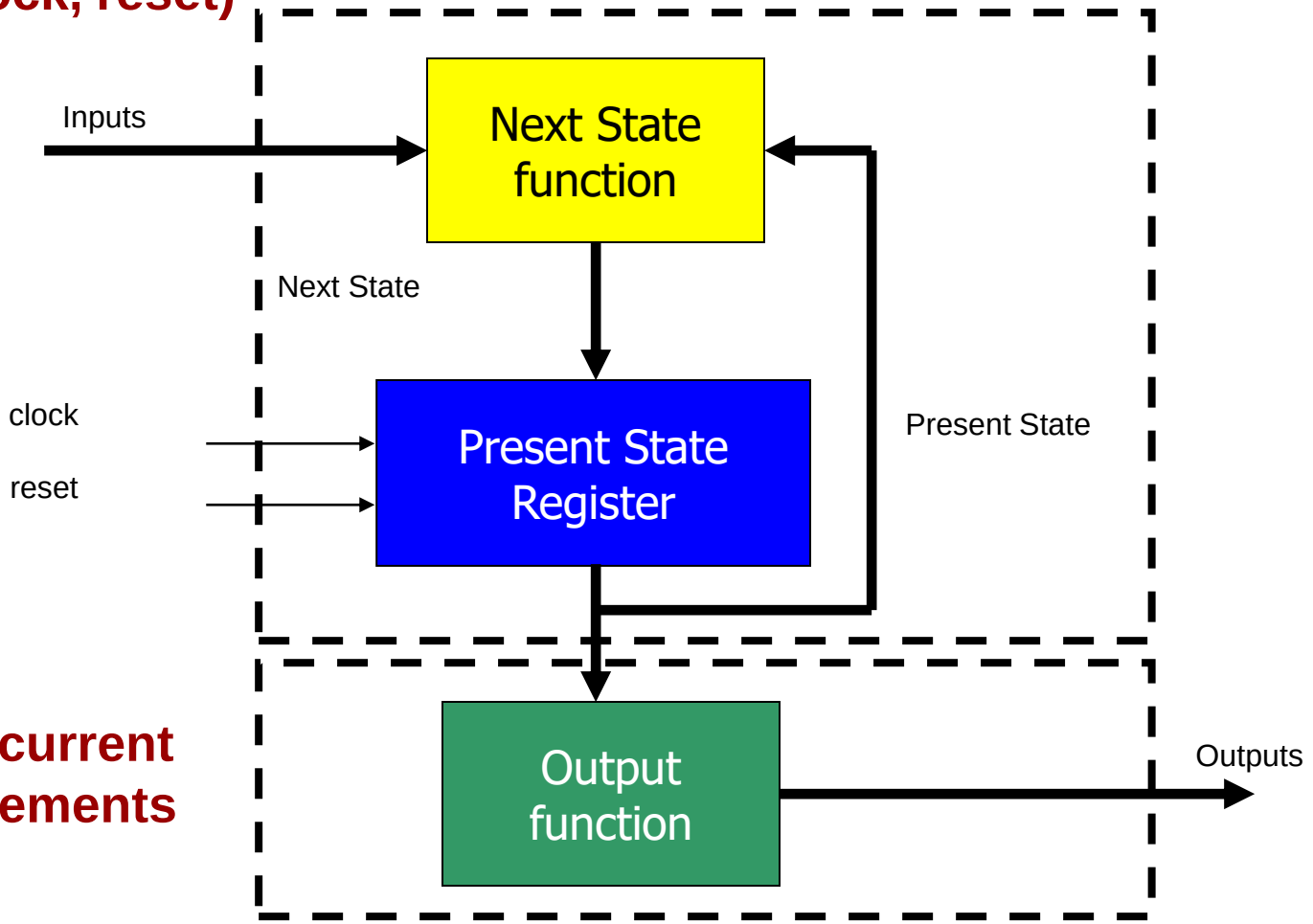
Module 7H - Alternate Coding Styles for Describing FSMs in VHDL

Alternate Moore and Mealy

- Alternate methods for writing Moore and Mealy FSMs in VHDL
- Sequence detector example

Moore FSM

process(clock, reset)



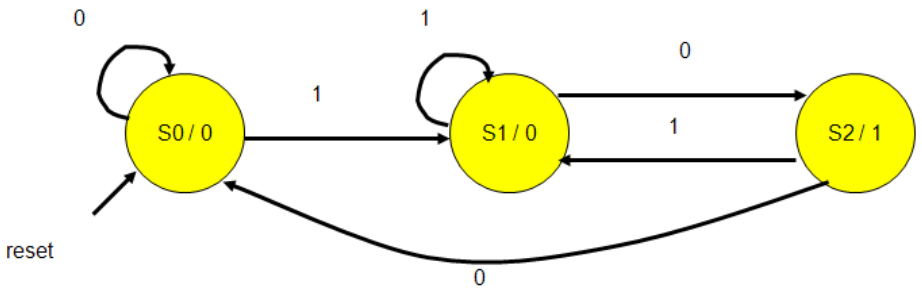
concurrent statements

Moore FSM in VHDL (Alternative Code)

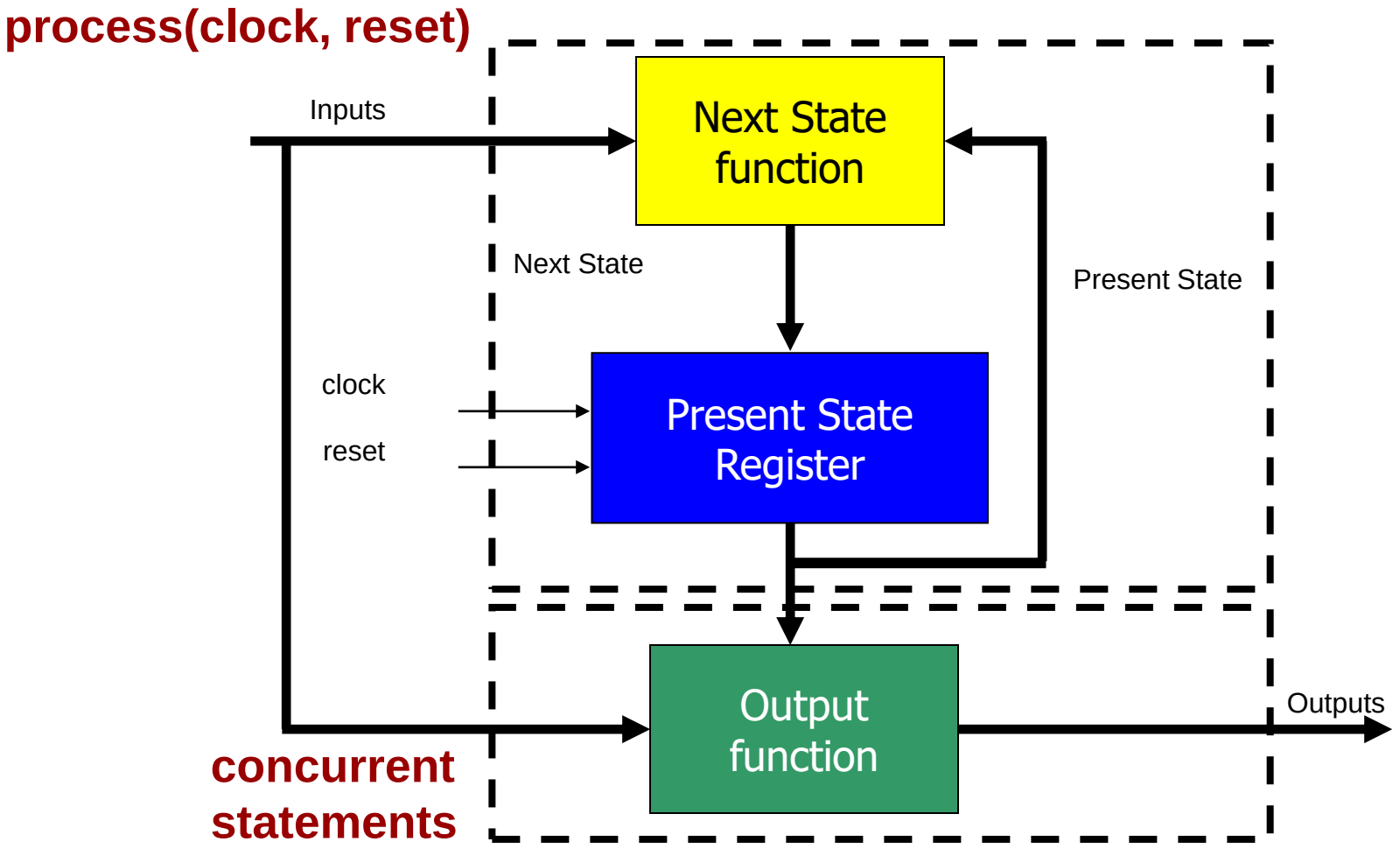
```
TYPE state IS (S0, S1, S2);
SIGNAL Moore_state: state;
SIGNAL next_Moore_state: state;

U_Moore: PROCESS (Moore_state, FSM_in)
BEGIN
   CASE Moore_state IS
   WHEN S0 =>
       FSM_out <= '0';
      IF FSM_in = '1' THEN
         next_Moore_state <= S1;
      ELSE
         next_Moore_state <= S0;
      END IF;
   WHEN S1 =>
       FSM_out <= '0';
      IF FSM_in = '0' THEN
         next_Moore_state <= S2;
      ELSE
         next_Moore_state <= S1;
      END IF;
   WHEN S2 =>
         FSM_out <= '1';
       IF FSM_in = '0' THEN
          next_Moore_state <= S0;
       ELSE
          next_Moore_state <= S1;
       END IF;
   END CASE;
END PROCESS;
```

```
PROCESS(clock, reset)
BEGIN
      IF reset = '1' then
         Moore_state <= S0;
      ELSIF rising_edge(clock) then
         Moore_state <= next_Moore_state;
      END IF;
END PROCESS;
```



Mealy FSM

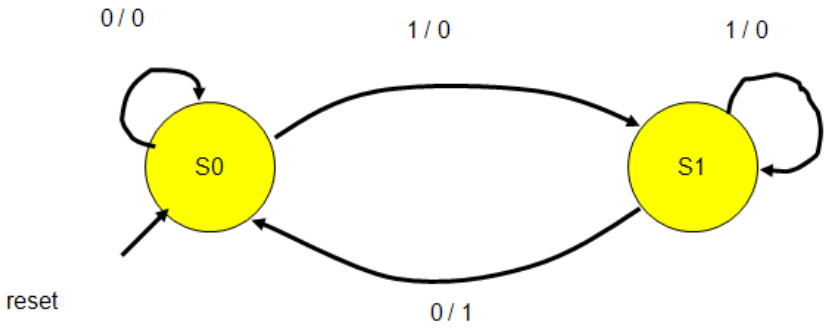

process(clock, reset)
Inputs
Next State function
Next State
Present State
clock
reset
Present State Register
Output function
Outputs
concurrent statements

Mealy FSM in VHDL (Alternative Code)

```
TYPE state IS (S0, S1);
SIGNAL Mealy_state: state;
SIGNAL next_Mealy_state: state;

U_Mealy: PROCESS(Mealy_state, FSM_in)
BEGIN
   CASE Mealy_state IS
   WHEN S0 =>
      IF FSM_in = '1' THEN
         FSM_out <= '0';
         next_Mealy_state <= S1;
      ELSE
         FSM_out <= '0';
         next_Mealy_state <= S0;
      END IF;
   WHEN S1 =>
      IF FSM_in = '0' THEN
         FSM_out <= '1';
         next_Mealy_state <= S0;
      ELSE
         FSM_out <= '0';
         next_Mealy_state <= S1;
      END IF;
   END CASE;
END PROCESS;
```

```
PROCESS(clock, reset)
BEGIN
     IF reset = '1' then
        Mealy_state <= S0;
     ELSIF rising_edge(clock) then
        Mealy_state <= next_Mealy_state;
     END IF;
END PROCESS;
```

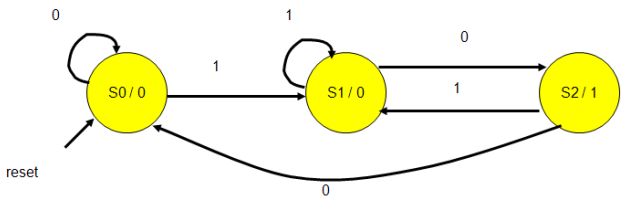


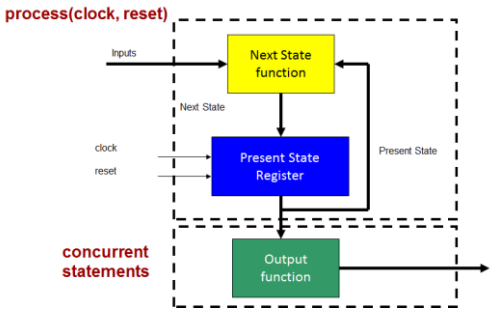
Moore FSM in VHDL

```
TYPE state IS (S0, S1, S2);
SIGNAL Moore_state: state;

U_Moore: PROCESS (clock, reset)
BEGIN
   IF(reset = '1') THEN
      Moore_state <= S0;
      FSM_out <= '0';
   ELSIF (rising_edge(clock)) THEN
      FSM_out <= '0'; -- Default
      CASE Moore_state IS
      WHEN S0 =>
         IF FSM_in = '1' THEN
            Moore_state <= S1;
         ELSE
            Moore_state <= S0;
         END IF;
      WHEN S1 =>
         IF FSM_in = '0' THEN
            Moore_state <= S2;
         ELSE
            Moore_state <= S1;
         END IF;
```

```
      WHEN S2 =>
         FSM_out <= '1';
         IF FSM_in = '0' THEN
            Moore_state <= S0;
         ELSE
            Moore_state <= S1;
         END IF;
      END CASE;
   END IF;
END PROCESS;
```





Mealy FSM in VHDL

```
TYPE state IS (S0, S1);
SIGNAL Mealy_state: state;

U_Mealy: PROCESS(clock, reset)
BEGIN
   IF(reset = '1') THEN
      Mealy_state <= S0;
      FSM_out <= '0';
   ELSIF (rising_edge(clock)) THEN
       FSM_out <= '0'; -- Default
      CASE Mealy_state IS
      WHEN S0 =>
         IF FSM_in = '1' THEN
            Mealy_state <= S1;
         ELSE
            Mealy_state <= S0;
         END IF;
      WHEN S1 =>
         IF FSM_in = '0' THEN
             FSM_out <= '1'
            Mealy_state <= S0;
         ELSE
            Mealy_state <= S1;
         END IF;
      END CASE;
   END IF;
END PROCESS;
```

